



Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla

TE3002B: Implementación de robótica inteligente (Gpo 501)

Dr. Alfredo García Suárez

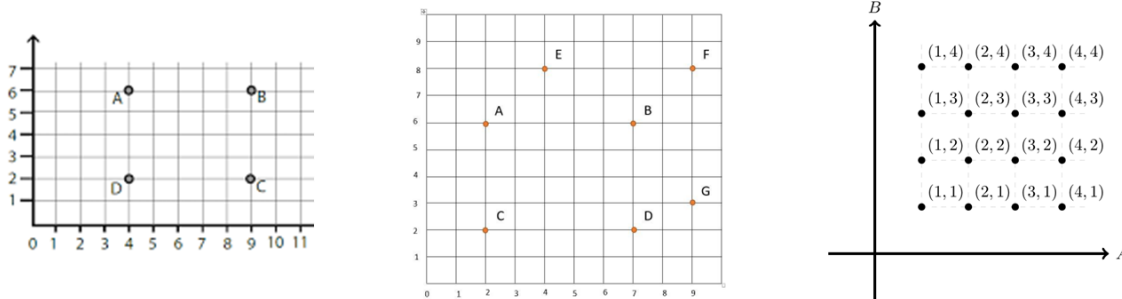
Actividad 8.1 (SLAM de Lidar)

Marcos Allen Martínez Cortés

A01737939

26 de mayo de 2026

Implementar el código requerido para generar el seguimiento de los siguientes waypoints de forma aleatoria, ajustando los parámetros: `sampleTime`, `tVec`, `initPose`, `lidar.scanAngles`, `lidar.maxRange`, waypoints, `controller.LookaheadDistance`, `controller.DesiredLinearVelocity` y `controller.MaxAngularVelocity`. Evadiendo los obstáculos del mapa de navegación “exampleMap” y “complexMap” (Para evadir los obstáculos es posible agregar way-points intermedios en el recorrido).



Procedimiento general

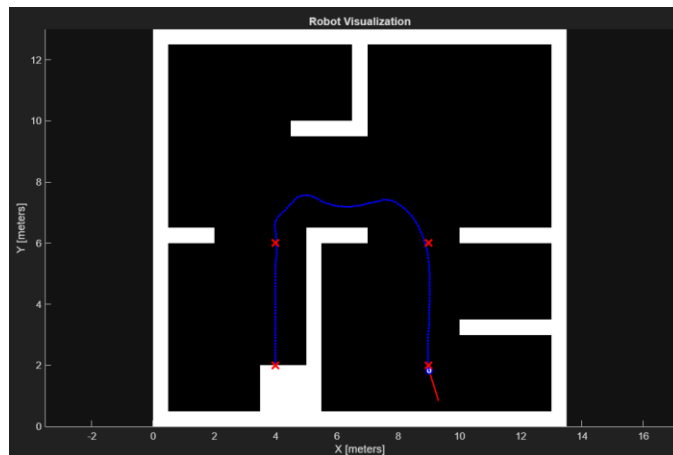
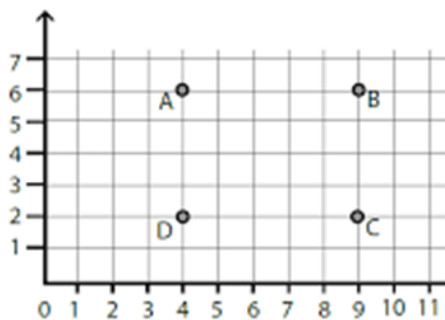
Para lograr el seguimiento de los waypoints de forma aleatoria, evadiendo los obstáculos de los mapas de navegación, “exampleMap” y “complexMap”, se siguió el siguiente procedimiento:

1. Planificación la ruta (orden de los waypoints), asegurando que se encuentren cercanos entre sí y evitando desde el inicio los obstáculos del mapa, más básicos para el “exampleMap” y más complejos, como su nombre lo indica, para el “complexMap”.
2. Establecer la pose (posición y orientación) del robot con base en el primer waypoint de la ruta diseñada.
3. Sintonizar el Pure Pursuit Controller, modificando los siguientes parámetros:
 - LookaheadDistance: es el parámetro más crítico, ya que, si es muy bajo, el robot intentará seguir la línea agresivamente provocando oscilaciones o giros sobre su propio eje. Por otra parte, si es muy alta, el robot recortará las esquinas y no pasará con precisión por los waypoints. Un valor de 0.5 podría ser un buen valor inicial, el cual se debería ir modificando.
 - DesiredLinearVelocity y MaxAngularVelocity: la velocidad lineal debe mantenerse moderada, ya que, si el robot va muy rápido, el robot no tendrá tiempo de reaccionar ante curvas cerradas u obstáculos. Por otra parte, es importante mantener una combinación de una velocidad lineal baja y una velocidad angular máxima alta, como 20 que venía en el código de ejemplo, que es lo que permite que el robot pueda reaccionar ante curvas cerradas y no colisionar.
4. Sintonizar el Lidar, modificando los siguientes parámetros:

- **maxRange:** es el rango máximo que ve el Lidar, esto le permite al robot ver a distancia y empezar a corregir la trayectoria mucho antes de llegar al obstáculo, aunque un valor alto hará que, si un waypoint está cerca de un obstáculo, el robot no pase por él, ya que estará reaccionando con el Lidar para evitar colisionar.
 - **scanAngles:** es el campo de visión y la resolución del Lidar en la simulación; a través de dos ángulos determina la cobertura de su visión, el cual en el código está para una cobertura 360° , y la resolución, el tercer campo de 200, indica la cantidad de puntos en los que se divide ese rango.
Con este, se puede establecer el alcance angular y junto con el maxRange, que sería la magnitud, se puede configurar qué “ve” el Lidar.
5. Cuando sea necesario, agregar waypoints intermedios según el comportamiento; sobre todo en el complexMap, añadir puntos que reduzcan la distancia entre puntos o el cambio de dirección brusco, le permite al robot tener un comportamiento adecuado.
 6. Modificar el tiempo de simulación para que el robot pueda completar el seguimiento de los waypoints.

Primer conjunto de waypoints

A) ExampleMap



Para este conjunto de waypoints en el exampleMap, se utilizaron los waypoints asignados para construir una ruta, de forma que se evitaran los puntos ciegos y los obstáculos del mapa.

Posteriormente se sintonizaron los parámetros del controlador, asignando:

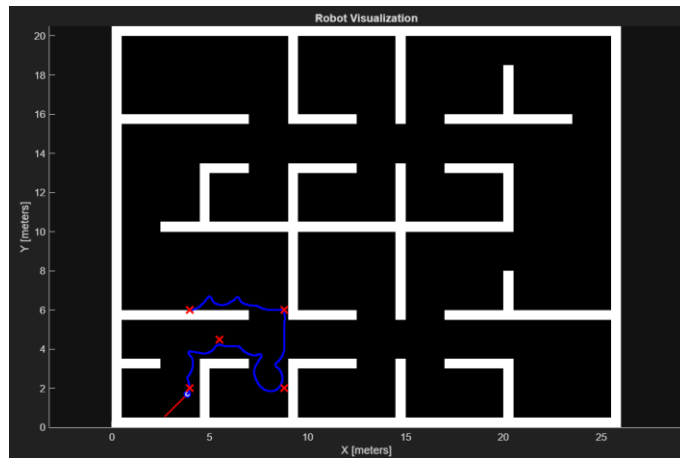
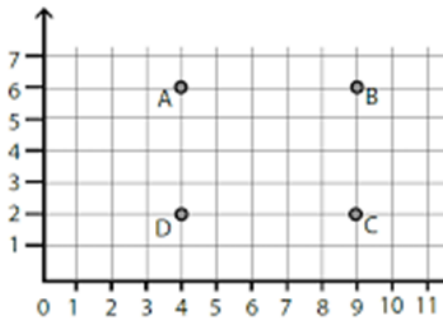
- **LookaheadDistance** de **1**, para que el robot pudiera ver más lejos, y tuviera una trayectoria más suave.

- DesiredLinearVelocity de **1**, que era el valor del código ejemplo, y que permite al robot avanzar a una velocidad considerable, pero con suficiente tiempo de reacción.
- MaxAngularVelocity de **20**, que era el valor del código de ejemplo y que en combinación permitieron que el robot pudiera avanzar y durante el trayecto, reaccionar a tiempo a giros bruscos por el cambio de dirección al seguir a los waypoints o para evitar colisionar con los obstáculos del mapa.

Se utilizó el valor del código de ejemplo de maxRange del Lidar de **1**.

Finalmente, se cambió el tiempo de simulación a **15 segundos**, para que el robot pudiera completar la ruta asignada con todos los waypoints.

B) ComplexMap



Para el complexMap, se siguió un procedimiento similar, y en la práctica se vio que se necesitaría de un waypoint intermedio para que el robot pudiera salir de una zona rodeada. De este modo se utilizó como waypoint intermedio: (5.5, 4.5)

Posteriormente se sintonizaron los parámetros del controlador, asignando:

- LookaheadDistance de **0.3**, para que el robot pasará con mayor precisión por los waypoints principales (es decir, no era necesario que pasara con precisión por el intermedio).
- DesiredLinearVelocity de **0.5** y MaxAngularVelocity de **20**, que en combinación permitieron que el robot pudiera avanzar y durante el trayecto, reaccionar a tiempo a giros bruscos por el cambio de dirección al seguir a los waypoints o para evitar colisionar con los obstáculos del mapa.

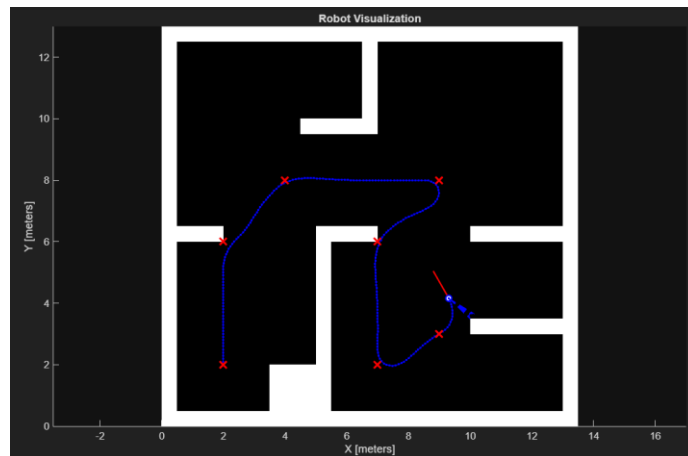
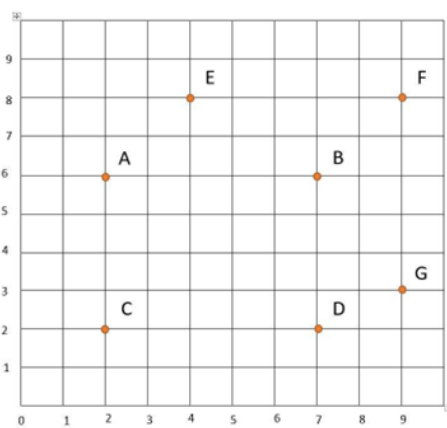
Se redujo el valor de maxRange del Lidar a **0.6**, ya que los waypoints estaban cerca de los obstáculos; por lo que era necesario que el valor fuera más bajo para que el robot pasara por

el waypoint y no se alejará en inmediato por el obstáculo, es decir que no reaccionara con mucha antelación.

Finalmente, se cambió el tiempo de simulación a **38 segundos**, para que el robot pudiera completar la ruta asignada con todos los waypoints principales, y el intermedio.

Segundo conjunto de waypoints

A) ExampleMap



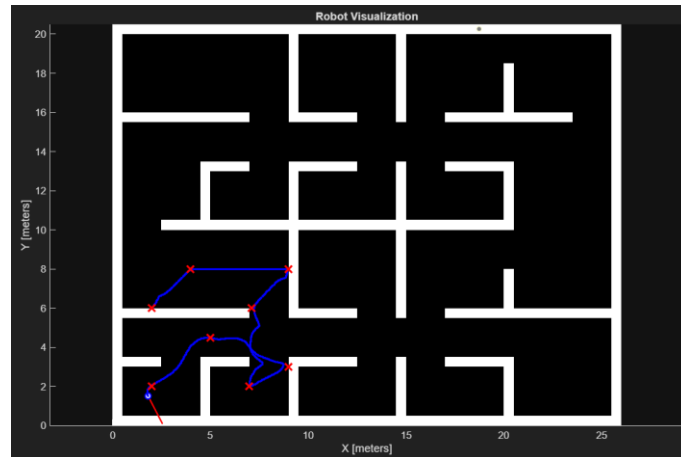
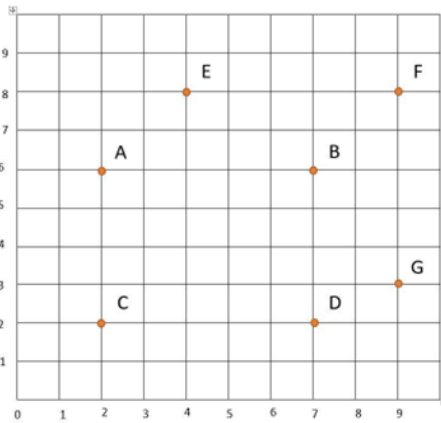
Para este conjunto de waypoints en el exampleMap, se utilizaron los waypoints asignados para construir una ruta, de forma que se evitaran los puntos ciegos y los obstáculos del mapa.

Posteriormente se sintonizaron los parámetros del controlador, asignando:

- LookaheadDistance de **0.75**, para que el robot tuviera un equilibrio entre suavidad de su trayectoria y precisión al pasar por los waypoints.
- DesiredLinearVelocity de **1** y MaxAngularVelocity de **20**, valores utilizados para el exampleMap del conjunto de waypoints anterior.

Se utilizó el valor del código de ejemplo de maxRange del Lidar de **1**. Finalmente, se cambió el tiempo de simulación a **22 segundos**, para que el robot pudiera completar la ruta asignada con todos los waypoints.

B) ComplexMap

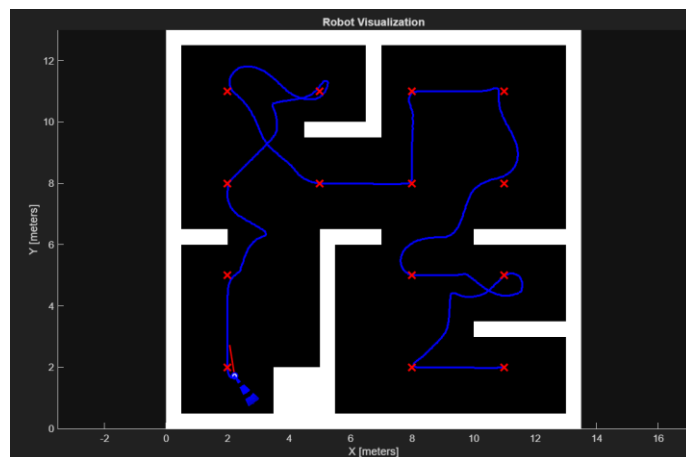
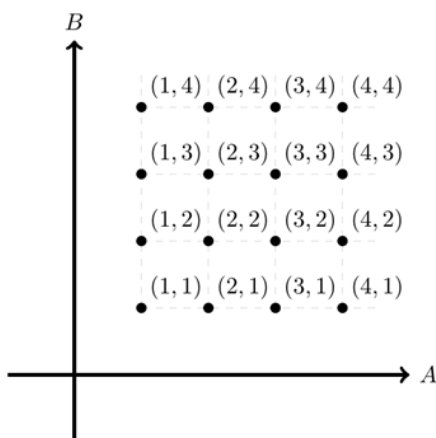


Para el complexMap, se siguió un procedimiento similar, y en la práctica se vio que se necesitaría de un waypoint intermedio para que el robot pudiera llegar de la zona rodeada al waypoint final. De este modo se utilizó como waypoint intermedio: (5, 4.5)

Posteriormente se sintonizaron los parámetros del controlador, y se utilizaron los mismos valores que en el primer conjunto de waypoints para el complexMap; de este modo se asignó LookaheadDistance de **0.3**, DesiredLinearVelocity de **0.5**, MaxAngularVelocity de **20**, y maxRange del Lidar a **0.6**. Finalmente, se cambió el tiempo de simulación a **52 segundos**.

Tercer conjunto de waypoints

A) ExampleMap



Para este conjunto de waypoints en el exampleMap, lo primero que se hizo fue re-escalar el conjunto de waypoints, siguiendo su patrón, para que se distribuyeran de forma adecuada en el mapa siguiendo su estructura de cuadrícula. De este modo, se utilizaron estos waypoints

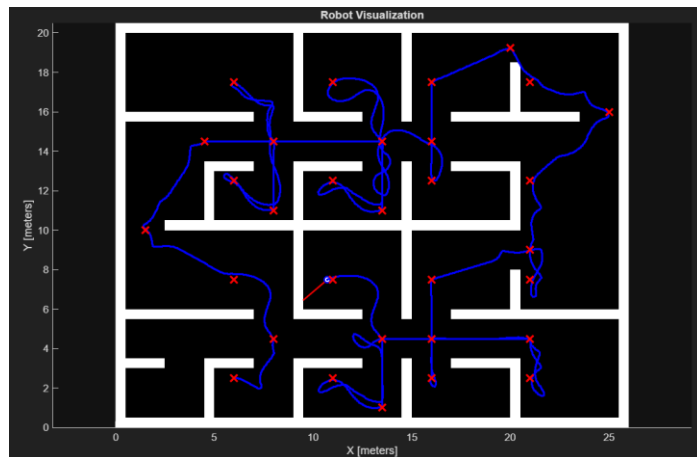
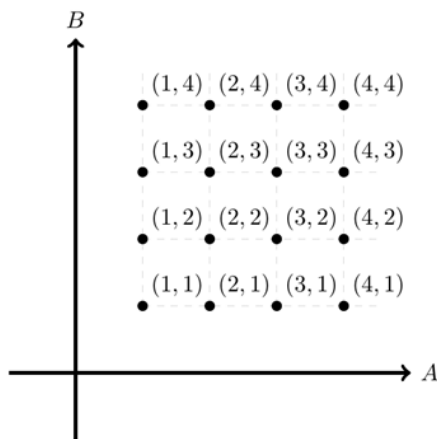
para construir una ruta, de forma que se evitaran los puntos ciegos y los obstáculos del mapa; debido a esto se omitieron dos waypoints que coincidían con obstáculos del mapa

Posteriormente se sintonizaron los parámetros del controlador, asignando:

- LookaheadDistance de **0.275**, para que el robot pasara con precisión por los waypoints.
- DesiredLinearVelocity de **0.5** y MaxAngularVelocity de **20**, que en combinación permitieron que el robot reaccionará ante cambios bruscos de dirección por la propia ubicación de los waypoints o por los obstáculos.

Se aumentó el valor del maxRange del Lidar a **1.2**, para que el robot pudiera ver más lejos y tuviera un mayor tiempo de reacción ante los obstáculos. Finalmente, se cambió el tiempo de simulación a **103 segundos**, para que el robot pudiera pasar por los waypoints.

B) ComplexMap



Para el complexMap, se siguió un procedimiento similar a los anteriormente indicados. Lo primero que se hizo fue re-escalar el conjunto de waypoints, para que se distribuyeran de forma adecuada en el mapa siguiendo su estructura de cuadrícula; aunque a diferencia del exampleMap, aquí no se omitieron waypoints. Por otra parte, en la práctica se emplearon varios waypoints intermedios, los cuales se consideraron necesarios por la gran cantidad de obstáculos y algunos puntos ciegos que se presentaron; estos se pueden encontrar en el código.

Posteriormente se sintonizaron los parámetros del controlador, asignando:

- LookaheadDistance de **0.3**, para que el robot pasará con mayor precisión por los waypoints principales.
- DesiredLinearVelocity de **0.65** y MaxAngularVelocity de **20**, que en combinación permitieron que el robot pudiera avanzar y durante el trayecto, reaccionar a tiempo.

Se utilizó el valor del código de ejemplo de maxRange del Lidar de **1**. Finalmente, se cambió el tiempo de simulación a **268 segundos**.